Your Signals Project study guide

CCEN 302 | Baseline analysis + AA-01 to AA-03 | 25 October 2026

Read this as a guided explanation of **the work actually completed**, not a claim that the entire course project is finished. It includes Yazan's baseline analysis and Abdulrahman's verified 22.05 kHz sampling and alignment work.

> **The result you should be able to explain**
> We reduced a 44.1 kHz recording to 22.05 kHz through filtered and unfiltered paths, reproduced both paths in Simulink, and compared them at matching physical times.

Contributor / area	What is present at this checkpoint
Yazan: original analysis	Audio collection; waveform, FFT, dB and spectrogram code; baseline Simulink model.
Abdulrahman: AA-01 to AA-03	Shared data format; one reduced-rate case; MAT/WAV handoff; reconstruction and MATLAB/Simulink checks.
Remaining team work	Other rates, full quantization/metric matrix, processed-signal comparisons, report, demonstration and final reviews.

How to study it

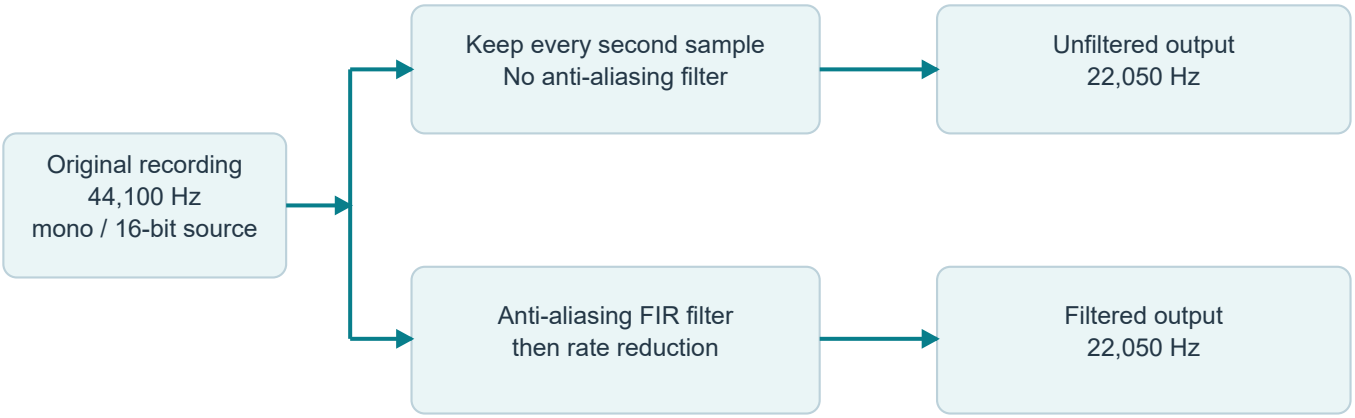
First read pages 2-7 for the signal-processing story. Read pages 8-11 for graphs and Yazan's analysis. Use pages 12-19 to trace the implementation, files and verified results. Attempt pages 20-21 before reading the answers on pages 22-23.

At each code block, ask: **What is the input? What changes? What stays unchanged? What units does each variable use?**

Scope: repository checkpoint be17af9. This guide does not claim that the microphone provenance, full submission, or other teammates' unfinished components have been verified. Source map and further reading: page 24.

1. The experiment in one picture

The two branches differ before samples are discarded.



Both branches start from the **same original recording**. That makes a filtered-versus-unfiltered comparison meaningful: we are changing the processing, rather than changing the input.

Question	Tool / representation
When is the sound strong or quiet?	Waveform: amplitude against time.
Which frequencies occur overall?	FFT magnitude spectrum: magnitude against frequency.
When do frequencies occur?	Spectrogram: time, frequency and color-coded strength.
Did MATLAB and Simulink implement the same case?	Compare aligned sample arrays from the two platforms.
How much did processing degrade the original?	Later NMSE/SNR and listening evaluation against the original reference.

The project files have distinct jobs

FirstPart_AudioFreqAnalysis.m is Yazan's baseline analysis. **Sampling_Abdulrahman.m** processes, plots and exports the prototype. **SamplingModel_Abdulrahman.slx** implements the two Simulink sampling branches. **Compare_Sampling.m** checks the model output after the simulation has run.

> **Check your understanding**
> A small difference between MATLAB and Simulink tests agreement between implementations. It is a different question from how closely either output resembles the original audio.

2. Samples, time and bit depth

Separate quantities that sound similar but mean different things.

Quantity	Meaning	Current value
Sampling rate Fs	Measurements stored per second	44,100 samples/s originally
Sound frequency f	Oscillation cycles per second	For example, a 300 Hz test tone
Sample count N	Number of stored measurements	882,005 in SpeechMusic.wav
Channel count	Independent audio channels	1: mono
Source bit depth	PCM precision in the recording	16 bits per sample
MATLAB data type	Representation after audioread	double-valued sample arrays

Two useful time calculations

```
duration = N / Fs;
time = (0:N-1) / Fs;
```

For the original recording, N/Fs is approximately **20.000113379 s**. The first sample is at t=0, and the last sample is at (N-1)/Fs, approximately **20.000090703 s**. The last timestamp and the sample-count duration differ by one sample interval.

A 1 kHz component at 44.1 kHz sampling has 44.1 measurements per cycle. At 22.05 kHz sampling it has 22.05 measurements per cycle. The sound frequency did not automatically become 500 Hz; correct rate conversion preserves its physical time scale.

> **Important distinction**
> Loading a 16-bit WAV into a double array does not invent extra recording accuracy. It changes the numeric representation used for calculations. Filtering can then produce values between the original PCM levels.

Yazan's waveform plot

```
numSamples = length(originalAudio);
time = (0:numSamples - 1) / Fs;
plot(time, originalAudio)
```

The time vector gives each stored sample its physical timestamp. Large excursions show stronger instantaneous sample amplitudes; quiet regions stay closer to zero. Positive and negative samples describe the oscillation around zero, not positive and negative loudness. The waveform locates events in time; the spectrum identifies their frequency components.

3. Loading and selecting samples

This is the first part of the actual MATLAB sampling script.

```
audioFile = "../Audios/SpeechMusic.wav";
[x, Fs] = audioread(audioFile);
```

Set MATLAB's **Current Folder to Matlab**. The path **..** goes up to the repository root, then enters Audios. The relative path is interpreted from the Current Folder; it is not automatically based on the saved script location.

Variable / expression	Meaning
x	Original audio samples as one column.
Fs	Sampling rate returned by the file reader: 44,100 Hz.
size(x)	882005-by-1: samples down the rows, one channel.
x(1:2:end)	Start at MATLAB index 1, step by 2, stop at the final valid index.

MATLAB indexes start at 1

```
x = [10; 20; 30; 40; 50];
x(1:2:end)     % gives [10; 30; 50]
```

The first retained value is the original first sample. Offset 0 in the Simulink Downsample block corresponds to this selection. MATLAB's **end** means the last available index, not a literal value stored in the signal.

> **A mistake we corrected**
> x(0:1:-1) does not select alternate samples. The colon expression is empty, and MATLAB indexing is one-based. The intended pattern is x(1:2:end).

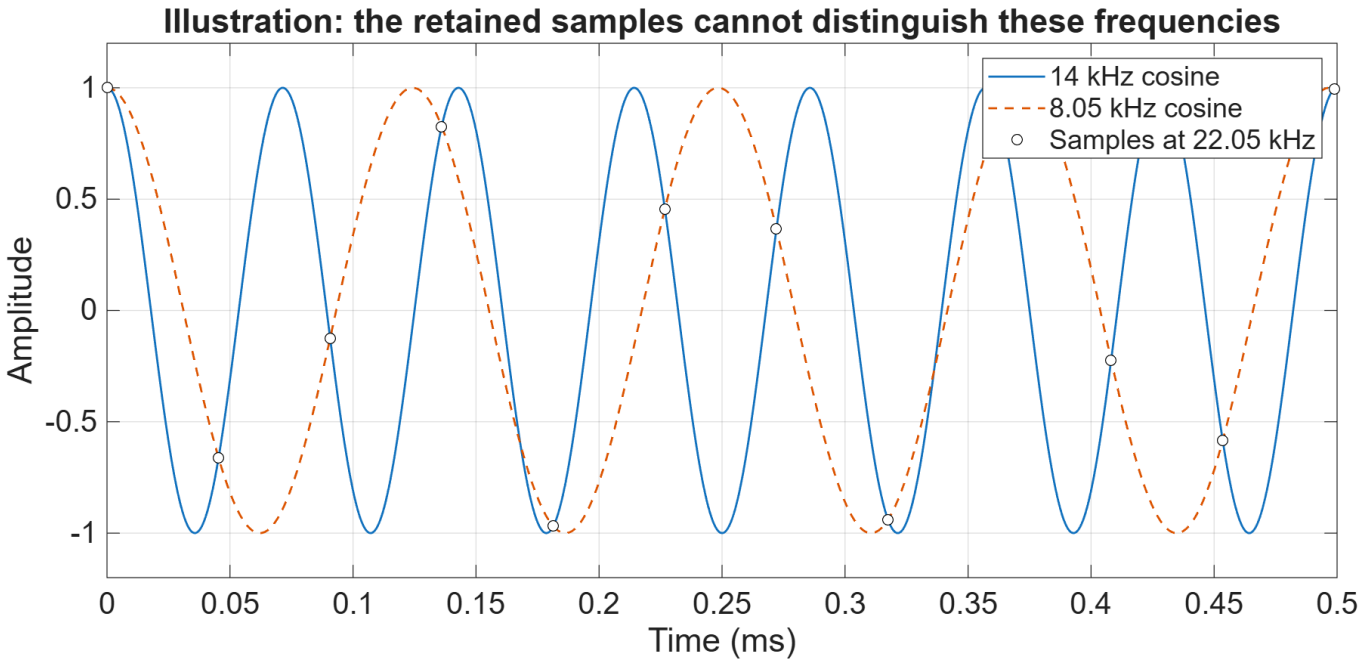
Related debugging lesson: one failed audioread leaves x undefined. Errors in later sections can then be consequences of that first failure, rather than separate processing bugs.

4. Why aliasing happens

A lower sampling rate can represent a smaller frequency range.

For a band-limited signal, faithful reconstruction requires a sampling rate greater than twice the highest retained signal frequency. The **Nyquist boundary** is Fs/2. A practical filter needs a transition region; treat the boundary as a limit, not a perfect passband specification.

Rate	Nyquist boundary
44,100 Hz	22,050 Hz
22,050 Hz	11,025 Hz
8,000 Hz	4,000 Hz



Teaching illustration, not a measured result from SpeechMusic.wav. Cosines at 14 kHz and 8.05 kHz give the same retained sample values at Fs=22.05 kHz.

At the new rate, a 14 kHz component can appear at **22.05 - 14 = 8.05 kHz**. The samples alone cannot tell whether it was originally 14 kHz or genuinely 8.05 kHz. This frequency ambiguity is aliasing.

> **Why the filter must come first**
> Suppress frequencies the new rate cannot represent before discarding samples. Filtering afterward cannot reliably separate aliased content from genuine content already at the same lower frequency.

5. The two MATLAB sampling paths

Only one rate reduction is completed so far: 44.1 to 22.05 kHz.

```
yNoFilter = x(1:2:end);
FsNoFilter = Fs / 2;
```

This path directly retains alternate samples. It contains **no anti-aliasing low-pass filter**. Merely changing Fs without changing the samples would instead change the interpreted playback speed.

```
filterDesignN = 10;
kaiserBeta = 5;
[yFiltered, filterCoefficients] = ...
    resample(x, 1, 2, filterDesignN, kaiserBeta);
FsFiltered = Fs / 2;
```

resample(x,p,q,...) changes the rate by p/q. Here 1/2 halves it. The function filters before reduction and compensates its designed filter delay. Start from x, not from yNoFilter: filtering after discarding samples would be too late to prevent aliasing.

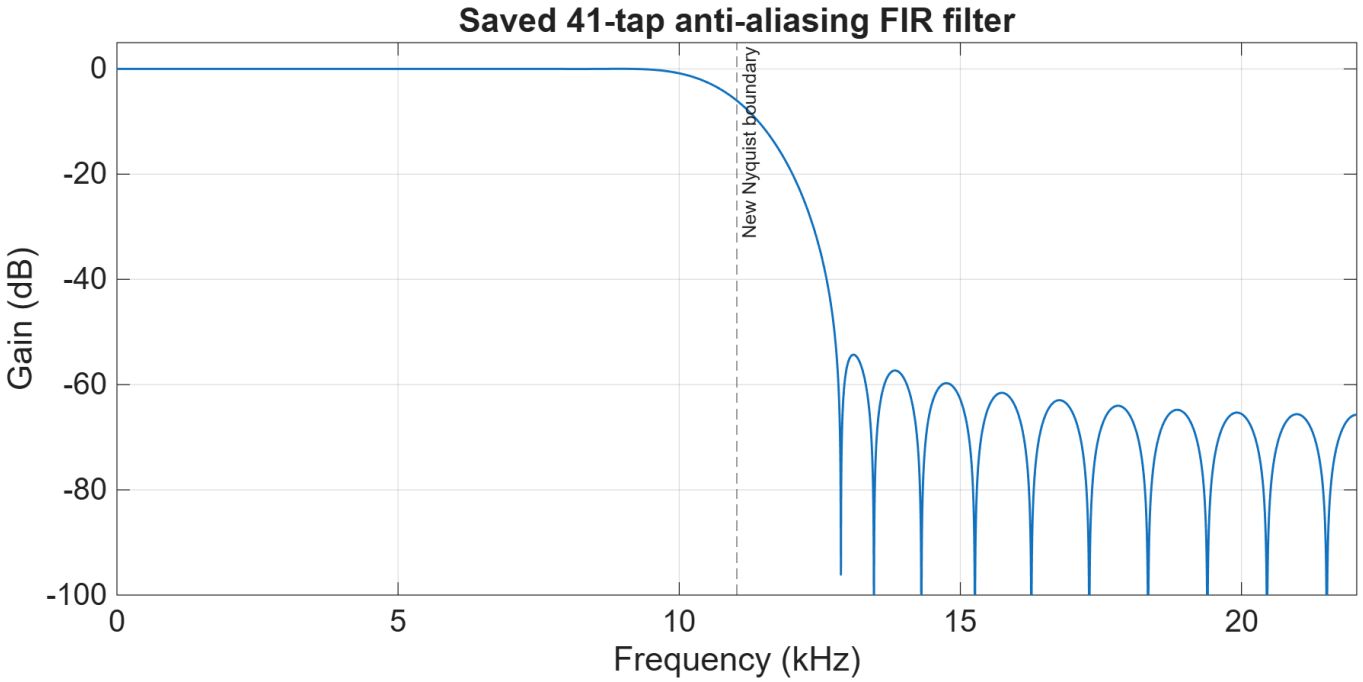
Signal	Samples	Rate	N/Fs duration
Original x	882,005	44,100 Hz	20.000113379 s
yNoFilter	441,003	22,050 Hz	20.000136054 s
yFiltered	441,003	22,050 Hz	20.000136054 s

For an odd N, selecting 1,3,5,... produces ceil(N/2) samples. The tiny duration difference is rounding at a sample boundary. It does not mean the recording was slowed down.

Code evidence: Sampling_Abdulrahman.m. Settings and exported counts: Results/Abdulrahman/AA02/summary.csv. API reference [S1].

6. FIR filtering and the Kaiser setting

The filter is finite and reproducible; it is not an ideal frequency wall.



Response calculated from the actual saved filter coefficients. The vertical line marks the new 11.025 kHz Nyquist boundary; it is not a claim that the passband is perfectly flat up to that frequency.

Term	What it means in this case
FIR	Finite impulse response: each output is a weighted combination of a finite number of input samples.
filterCoefficients	The 41 weights specifying the actual filter.
Order	40: one less than the number of coefficients.
filterDesignN = 10	resample design parameter; order = 2*n*max(p,q) = 40 here.
kaiserBeta = 5	Shape parameter for the Kaiser window used in the FIR design.

At fixed length, increasing Kaiser beta generally lowers window sidelobes while broadening the mainlobe. In filter design this trades unwanted-frequency suppression against transition width. It is not a separate processing stage, and a larger beta is not automatically better for every objective.

Two windows with different jobs
Kaiser participates in designing this anti-aliasing FIR filter. Yazan's Hamming window weights short audio segments for the spectrogram. They are not interchangeable settings in the same operation.

Implementation settings are verified in the saved MAT data. Window terminology: [S1, S2].

7. Yazan's FFT: what each line means

An FFT changes the representation from sample amplitudes to frequency coefficients.

```
N = length(originalAudio);
audioFFT = fft(originalAudio);
frequencies = (0:floor(N/2)) * Fs / N;
fftMag = abs(audioFFT / N);
fftMag = fftMag(1:floor(N/2) + 1);
```

These are the same operations used in the current baseline script; N is written here as a shorter name for numSamples. FFT coefficients are usually **complex**: they contain magnitude and phase information. abs extracts magnitude. Dividing by N gives normalized coefficients rather than raw FFT sums.

Operation	Interpretation
fft(originalAudio)	Compute N frequency coefficients.
Fs/N	Frequency spacing between adjacent DFT bins, in hertz.
0:floor(N/2)	Bin numbers beginning at DC (k=0).
abs(audioFFT/N)	Magnitude of each normalized complex coefficient.
1:floor(N/2)+1	MATLAB array positions corresponding to the retained bins.

Worked bin labels: N=8, Fs=8,000 Hz

The spacing is 8,000/8 = 1,000 Hz. The retained frequency labels are **[0, 1000, 2000, 3000, 4000]**. Their MATLAB positions are [1,2,3,4,5]. This is why the indexing expression contains +1: DC occupies a position too.

> **Keep the frequency axis tied to its signal**
> For yNoFilter, use FsNoFilter and length(yNoFilter). Reusing the original Fs would put the peaks at incorrect frequency labels.

Code evidence: FirstPart_AudioFreqAnalysis.m and Sampling_Abdulrahman.m. Technical reference [S3].

8. Why we double most FFT magnitudes

A one-sided amplitude spectrum combines the paired contributions of real audio.

For a real-valued recording, positive- and negative-frequency coefficients occur as complex-conjugate pairs. Their magnitudes match. When displaying only the nonnegative side, we double the paired magnitudes so a bin-centered sinusoid retains its expected amplitude.

```
if mod(N, 2) == 0
    fftMag(2:end-1) = 2 * fftMag(2:end-1);
else
    fftMag(2:end) = 2 * fftMag(2:end);
end
```

Sample count	Do not double	Double
Even N	DC and the exact Nyquist bin	Positions 2 through end-1
Odd N	DC only; no exact Nyquist bin exists	Positions 2 through end

The original primary file has **882,005** samples, and each downsampled file has **441,003**. Both counts are odd. Their last retained bin is slightly below Fs/2, although their Nyquist boundary is still Fs/2.

The parity bug that was fixed

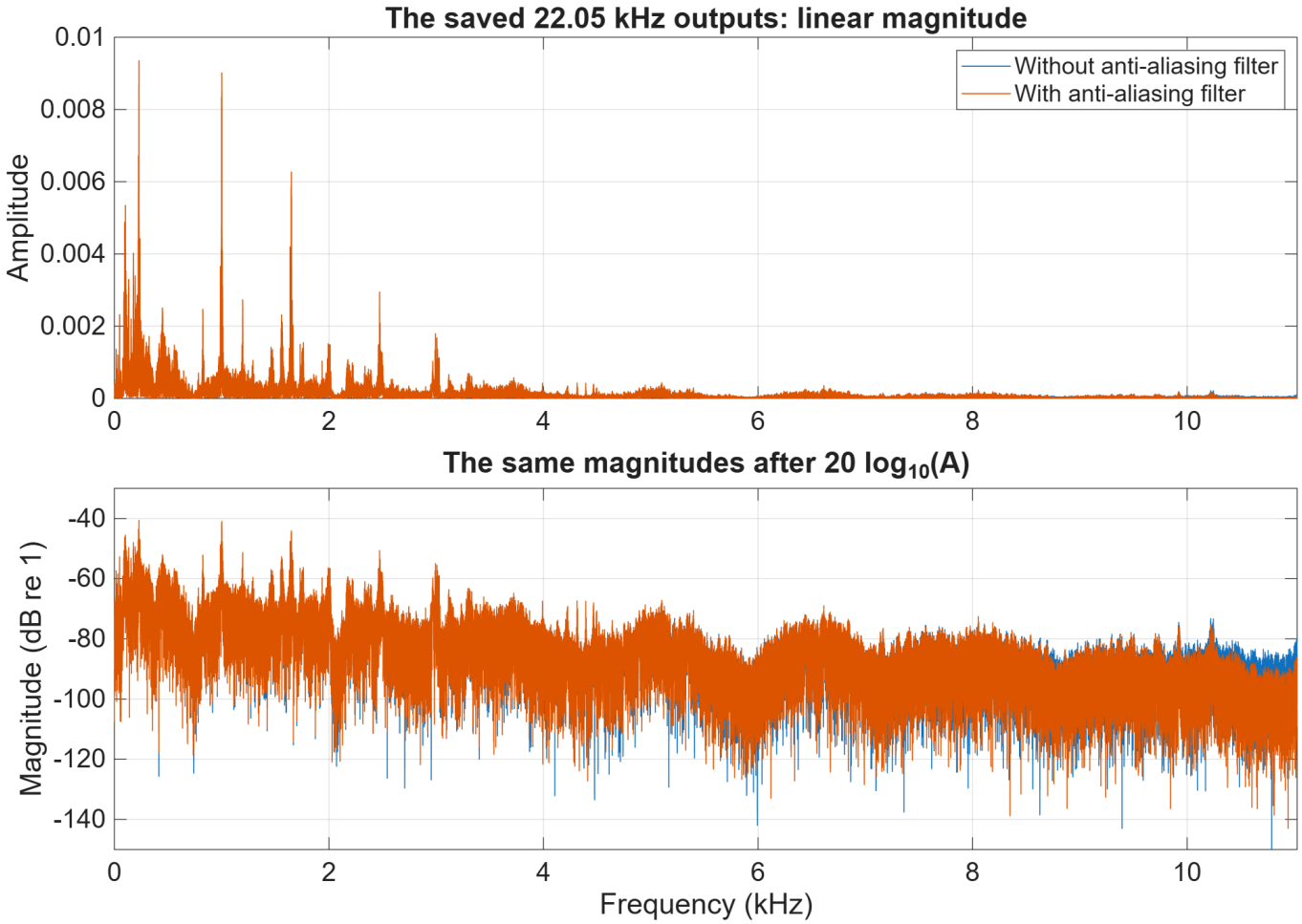
```
if numSamples%2==0        % wrong parity test in MATLAB
if mod(numSamples,2)==0    % intended even-count test
```

In MATLAB, % begins a comment. The first version therefore behaves like if numSamples; it does not calculate a remainder. mod(N,2) returns 0 for even counts and 1 for odd counts. Yazan's committed code now uses the proper condition.

> **Magnitude does not tell the whole story**
> Matching magnitude spectra do not establish identical waveforms. Phase and time alignment matter too. Our sample-by-sample MATLAB/Simulink comparison checks more than similar-looking plots.

9. Understanding the dB spectrum

The horizontal axis stays the same; only the magnitude scale changes.



Replotted from the exact saved AA-02 sample arrays. The lower panel uses a display floor only when taking logarithms; the processing data are unchanged.

```
fftMagDb = 20 * log10(fftMag);
```

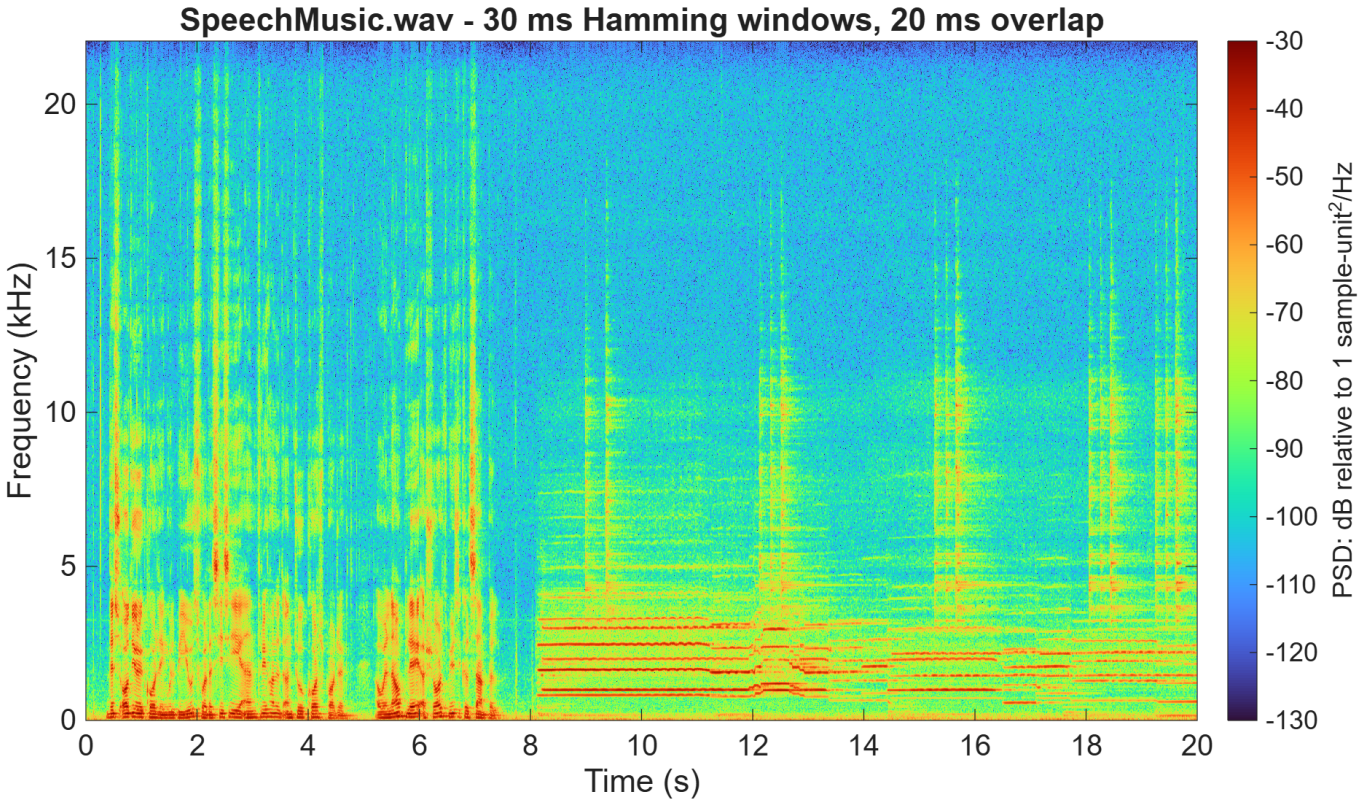
Here the implicit amplitude reference is 1. Magnitude 0.01 becomes -40 dB; 0.001 becomes -60 dB; 0.0001 becomes -80 dB. **A 20 dB increase means ten times the amplitude**. Thus -60 dB is stronger than -80 dB.

The linear scale makes weak components look close to zero. The logarithmic scale separates them visually, so high-frequency differences are easier to inspect. Negative dB means below the reference, not negative amplitude.

> **What this graph cannot prove**
> Close curves at one display scale do not prove that aliasing is absent. These digital spectrum levels are not calibrated microphone dB SPL. Also, log10(0) is -Inf; any plotting floor should not silently change the signals used for metrics.

10. Yazan's spectrogram

Short, overlapping FFT analyses show how frequency content changes with time.



Recomputed from the original recording using the tracked window/overlap/FFT settings. Color is power density in numeric sample units, not a calibrated acoustic measurement.

```
window = round(0.03 * Fs);
overlap = round(0.02 * Fs);
fftBins = 2048;
spectrogram(originalAudio, hamming(window), ...
            overlap, fftBins, Fs, "yaxis")
```

At 44.1 kHz, the window contains **1,323 samples**, overlap is **882 samples**, and the hop is 441 samples = **10 ms**. Each short segment is weighted by a Hamming window to reduce spectral leakage from its boundaries.

Horizontal bands indicate sustained frequency components. Moving bands indicate changing pitch. Vertical streaks indicate short broadband events. Color shows the strength at each time-frequency location; cursor lines and markers are measurement aids, not audio features.

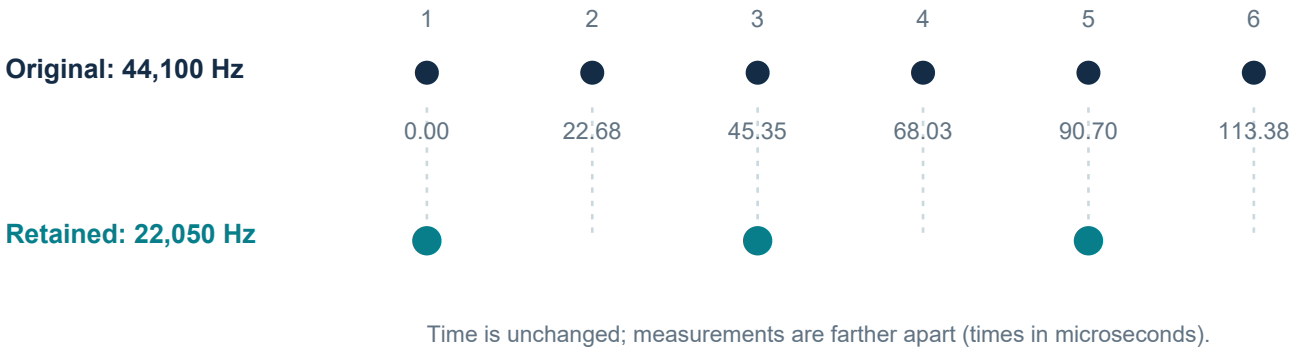
Resolution and labels

The FFT grid spacing is 44,100/2,048 = 21.533 Hz. Zero-padding a 1,323-sample window to 2,048 points gives a denser grid, not extra observed audio or equivalent physical resolution. A frequency axis in kHz does not automatically change density units from per Hz to per kHz.

Baseline code / model evidence [R1, R2]. STFT interpretation [S4]. Use the actual estimator units; handwritten colorbar text alone does not establish physical units.

11. Reconstruction and the common time grid

Equal physical times are necessary for a meaningful error calculation.



Time is unchanged; measurements are farther apart (times in microseconds).

At index 2, the original array represents t=1/44,100 s, but the downsampled array represents t=1/22,050 s. Directly subtracting their corresponding entries compares different moments. Equalizing lengths alone would not fix that time mismatch.

```
yReconstructed = resample(yFiltered, 2, 1);
FsReconstructed = Fs;
yAligned = yReconstructed(1:length(x));

yNoFilterReconstructed = resample(yNoFilter, 2, 1);
yNoFilterAligned = yNoFilterReconstructed(1:length(x));
```

Stage	Primary recording sample count
Original	882,005
Downsampled	441,003
Reconstructed at 44.1 kHz	882,006
Retained on original grid	882,005

Upsampling by 2 creates intermediate values through interpolation filtering. It restores the sampling grid, **not the high-frequency information lost or aliased earlier**. The extra reconstructed endpoint lies beyond the original grid and is excluded.

> **Why the crop is justified**
> Both grids start at zero and have the same 44,100 Hz rate. We retain the complete original interval. We do not discard poorly matching parts or shift the signal by whichever lag happens to minimize error.

12. Yazan's baseline Simulink model

The baseline shows the original input before the new sampling branches.

Block / chain	Purpose and saved setting
From Multimedia File	Read SpeechMusic.wav. Original rate 44,100 Hz; mono; 2,048 samples per audio frame.
Time graph	Display original amplitude over time.
FFT	Normalized FFT inherited from the 2,048-sample input frame.
Complex to Magnitude-Angle	Extract FFT magnitude for the spectrum display.
Selector	Retain entries 1:1025: DC through Nyquist for an even 2,048-point FFT.
Gain	[1; 2*ones(1023,1); 1]: leave DC/Nyquist alone and double interior magnitudes.
Array Plot	Frequency increment 21.533203125 Hz = 44,100/2,048.
Spectrogram	Power-density view, Hamming, 1,323-sample window, 2,048-point FFT, about 66.7% overlap.

Why the MATLAB and Simulink spectra can look different

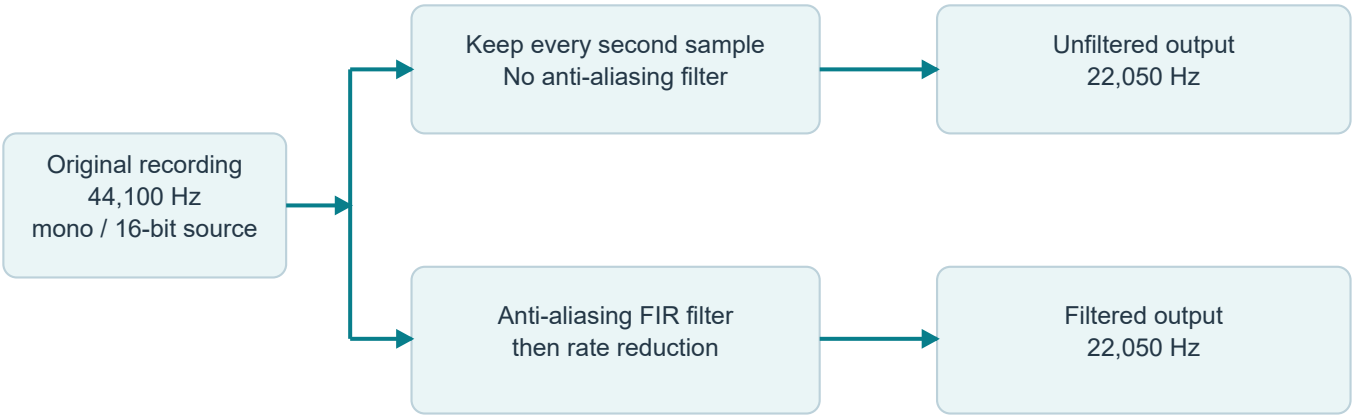
Yazan's MATLAB FFT uses the whole recording, while the saved Simulink FFT processes successive 2,048-sample frames. They have different observation intervals and bin spacings. A graph of the current short frame need not equal the full-record FFT.

Compare equivalent analysis settings when drawing numerical conclusions. Normalization, amplitude versus power, windowing, averaging, units, and the selected time interval all matter.

Credit and verification
The baseline implementation is Yazan's contribution. Our independent checks ran the new sampling model, which includes these baseline analysis branches. They do not establish microphone provenance or complete every baseline/report requirement.

13. The two new Simulink branches

Both must receive the original input directly.



Block	Configuration in the verified 22.05 kHz case
Downsample	Factor 2; offset 0; Columns as channels (frame based); Enforce single-rate processing; arbitrary frame length unchecked.
FIR Decimation	Dialog parameters; coefficients = filterCoefficients; factor 2; Direct form; frame-based; Enforce single-rate processing.
To Workspace: unfiltered	Variable simNoFilter; Timeseries; limit inf; decimation 1; inherited sample time -1.
To Workspace: filtered	Same logging settings, variable simFiltered.
Source playback	Number of times to play file = 1; keep inherited sample time and 2,048-sample frames.

FIR Decimation combines filtering and rate reduction. Do not add a second Downsample block after it for this case. Choosing Auto coefficients could design a different filter; using the saved vector makes the comparison controlled.

> **The single-rate label is about frames**
> Input frames arrive at the same interval after the block, but contain half as many samples: 2,048 becomes 1,024. Therefore the audio sample rate is halved. The block is not preserving the original 44.1 kHz audio rate.

Block behavior and parameter terminology: [S5, S6, S7].

14. Frames, timestamps and saved samples

A logged timestamp describes a whole frame update.

```
simNoFilter = out.simNoFilter;
size(simNoFilter.Data)
% 1024     1     431
```

Dimension	Meaning
1,024	Consecutive audio samples in each output frame.
1	One audio channel.
431	Frames logged during the simulation.

The first frame timestamps are approximately 0, 0.0464, and 0.0929 s. Use the unrounded difference between timestamps for calculations. One frame period is 2,048/44,100 = 1,024/22,050, approximately **0.046439909 s**.

```
framePeriod = simNoFilter.Time(2) - simNoFilter.Time(1);
FsSimNoFilter = size(simNoFilter.Data,1) / framePeriod;
simNoFilterSamples = simNoFilter.Data(:);
```

The rate calculation returns 22,050 Hz. The (:)-indexing stacks frame 1's samples, then frame 2's, and so on into one column. It does not average the samples or change their values.

Count	Interpretation
431 * 1024 = 441,344	All samples in the logged complete frames.
441,003	Samples covering the primary processed recording.
341 extra samples	Final-frame coverage beyond the desired recording interval.

Do not confuse the two rates
1/framePeriod is the frame-update rate, about 21.533 frames/s. Multiplying by 1,024 samples/frame gives the audio rate, 22,050 samples/s. To Workspace decimation 1 means log every update; it is not another audio downsampling factor.

15. Known FIR delay and comparison

Same coefficients alone do not guarantee samples appear at the same time index.

The saved symmetric linear-phase FIR has 41 taps. Its raw group delay is (41-1)/2 = **20 input samples**. With factor-2 decimation, this is **10 output samples**, or approximately 0.4535 ms.

MATLAB resample compensates this delay. The causal Simulink FIR Decimation output retains it. We compensate the known delay when comparing corresponding samples; we do not search for an error-minimizing shift.

```
simNoFilterSamples = out.simNoFilter.Data(:);
simFilteredSamples = out.simFiltered.Data(:);

delayInputSamples = (length(filterCoefficients)-1)/2;
delayOutputSamples = delayInputSamples/2;

simNoFilterAligned = simNoFilterSamples(1:length(yNoFilter));
simFilteredAligned = simFilteredSamples( ...
    delayOutputSamples+1 : delayOutputSamples+length(yFiltered));

maxDifferenceNoFilter = ...
    max(abs(simNoFilterAligned-yNoFilter));
maxDifferenceFiltered = ...
    max(abs(simFilteredAligned-yFiltered));
```

For the primary recording, the filtered range is **11:441013**. It skips 10 delayed output samples and keeps exactly 441,003 samples. The remaining final-frame padding is excluded.

> **Two different alignment operations**
> This page compares MATLAB and Simulink at 22,050 Hz. Page 12 reconstructs processed audio onto the 44,100 Hz original grid for later quality metrics. Do not compensate the FIR delay twice or confuse these two comparison grids.

This is the logic now isolated in Compare_Sampling.m. Changing rates later requires revisiting the delay/rate relationship rather than copying a hard-coded 10.

16. What the verification results prove

Implementation agreement and audio quality are different measurements.

Input	No-filter max difference	Filtered max difference
SpeechMusic.wav	0	3.33066907388e-16
300Hz_Sine.wav	0	6.93889390391e-17

The filtered differences are at floating-point rounding scale. This strongly supports that MATLAB and Simulink implement the same configured sampling operation over the compared recording interval.

The known-tone check confirmed a dominant 300 Hz frequency, 882,000 source samples, 441,000 processed samples, and zero residual reconstruction lag for both paths. The check used the full recording; a small lag search was diagnostic only, not an applied correction.

> **What is not proved**
> A 300 Hz tone lies well below 11.025 kHz, so it checks timing/consistency but does not demonstrate high-frequency folding. Near-zero cross-platform differences do not prove that downsampling left the original audio unchanged.

The later quality question

Let E_signal be the sum of squared original samples and E_error the sum of squared aligned original-minus-processed differences. For positive energies:

```
NMSE = E_error / E_signal
SNR_dB = 10 * log10(E_signal / E_error)
       = -10 * log10(NMSE)
```

An illustrative NMSE of 0.01 corresponds to SNR = 20 dB. These are teaching values, not reported project measurements. Zero error and zero-energy references need explicit handling. Jesher owns the quality metrics; the cross-platform max differences above are not NMSE/SNR results.

Measured evidence: known_tone_validation.csv and verified run recorded in team/Abdulrahman.md. The project brief specifies the later metric definitions.

17. MAT, WAV and the handoff metadata

One file preserves calculation data; the other can be played directly.

File	Contents / intended use
sampling_22050.mat	prototype structure: original and processed double arrays plus rates, timing and filter settings.
..._none_matlab.wav	Unfiltered 22.05 kHz, mono, 16-bit PCM playback copy.
..._lpf_matlab.wav	Filtered 22.05 kHz, mono, 16-bit PCM playback copy.
summary.csv	Case IDs, rates, counts/durations, filter and WAV metadata.
comparison.png	Saved linear spectrum comparison.
known_tone_validation.csv	Measured AA-03 known-tone checks, stored under Results/Abdulrahman/AA03.

```
data = load("Results/Abdulrahman/AA02/sampling_22050.mat");
prototype = data.prototype;
signal = prototype.filtered.samples;
signalFs = prototype.filtered.Fs;
sound(signal, signalFs);
```

The example above starts from the **repository root**. Passing the correct rate to playback matters: 22.05 kHz samples played as 44.1 kHz would run twice as fast.

The MAT arrays are retained exactly. WAV writing maps them to 16-bit PCM levels. The measured maximum round-trip error is 0 for the unfiltered copy and approximately **3.05173e-5** for the filtered copy. That is export conversion error, not total sampling degradation.

> **What b16 means here**
> The case label identifies a 16-bit source/control. The processed MAT arrays remain double-valued and have not undergone Khaled's reduced-bit-depth experiment. The separate WAV carrier is 16-bit; keep carrier precision and quantizer settings distinct.

18. Run order, bookkeeping and the checkpoint

Reproduce the result without relying on unsaved Command Window history.

Step / Current Folder	Run	Why
1. Matlab	Sampling_Abdulrahman	Creates MATLAB signals, filterCoefficients, plots and exports. Does not need out.
2. Repository root	SamplingModel_Abdulrahman.slx	Source path is Audios/SpeechMusic.wav. Run the model to create out.
3. Matlab	Compare_Sampling	Reads out and the MATLAB variables; prints both implementation differences.

Keep workspace variables between steps. Re-run the processing and model after changing the input, filter, or rate. An old out object can produce a misleading comparison against stale simulation results.

What the export code does

audiowrite / audioread: save WAV copies and measure their conversion error. **dir(...).bytes:** obtain actual file size. **struct:** group named metadata. **save:** preserve prototype in a MAT file. **table / writetable:** create a readable CSV. **exportgraphics:** save the figure. These steps package the experiment; they do not perform more downsampling.

Each output WAV contains 441,003 mono samples at 16 bits: 882,006 payload bytes. Its measured size is 882,050 bytes, including 44 bytes of header in these files. Other WAV layouts can have different header overhead.

Keep source and evidence; ignore temporary caches

Commit .m scripts, .slx models, intentional recordings/results, and task documents. .asv is an Editor backup; slprj/ and .slxc are generated Simulink caches; .slx.original is a local backup. Ignoring them keeps them out of future Git additions without deleting them.

> **Where we stop**
> AA-01 to AA-03 are complete for the first sampling case. AA-04 extends rates; AA-05 adds the full evidence set; later work includes combined quantization/metrics, report and demonstration. Git commit be17af9 is the source checkpoint for this guide.

Self-check A: signals and graphs

Attempt these before looking at the answer key on page 22.

1. What is the difference between a 1 kHz audio component and a 44.1 kHz sampling rate?

2. At 22,050 samples/s, what is the Nyquist boundary? What must happen to content above that boundary before downsampling?

3. Why can a 14 kHz component appear at 8.05 kHz after unfiltered reduction to 22.05 kHz?

4. Which original sample positions does x(1:2:end) retain? For N=5, how many samples remain?

5. Why are the two output durations approximately, rather than necessarily exactly, equal to the original N/Fs duration?

6. What extra processing does resample(x,1,2) perform compared with alternate-sample indexing?

7. For N=8 and Fs=8,000 Hz, list the one-sided FFT frequency labels. Why are there five labels?

8. Which FFT magnitudes are doubled for even N? How does odd N change the last retained bin?

9. Which component has the larger amplitude: -60 dB or -80 dB? By what factor? Is -80 dB negative amplitude?

10. In a spectrogram, what can horizontal bands and vertical streaks indicate? Why does a 2,048-point FFT not by itself guarantee 21.533 Hz physical resolving power?

> **Explain without reading code**
> Describe why the two branches must receive the same original signal, and why applying the anti-aliasing filter after sample deletion is too late.

Self-check B: implementation and handoff

Attempt these before looking at the answer key on page 23.

11. A 44.1 kHz source has 882,005 samples. Explain the sequence 882005 -> 441003 -> 882006 -> 882005.

12. Why does interpolation back to 44.1 kHz not undo aliasing or restore lost high frequencies?

13. What does a saved data size of 1024-by-1-by-431 mean? How many stored audio sample values does it contain?

14. Frame timestamps differ by approximately 0.046439909 s. Why is the audio rate 22,050 Hz rather than about 21.533 Hz?

15. A symmetric 41-tap FIR precedes factor-2 decimation. Calculate its delay in input and output samples. Why start the aligned filtered range at index 11?

16. MATLAB and Simulink differ by about 3.33e-16. Does that establish excellent audio quality relative to the original? Explain.

17. Which file should preserve exact filtered double samples: the MAT or the 16-bit WAV? Why does the filtered WAV have a small round-trip error?

18. Why can the complete Sampling_Abdulrahman script run before Simulink, while Compare_Sampling cannot?

19. State the agreed comparison interval and explain why removing the extra endpoint differs from trimming poorly matching signal regions.

20. Name two generated files/folders that should be ignored by Git, and two intentional project artifacts that should remain tracked.

A useful oral-defense test: explain your whole signal path in 90 seconds, including the meaning of the sampling rates, filtering, logged frames, known delay, and the measured differences.

Answer key A

Compare your reasoning, not just the final number.

1. Sound frequency counts cycles per second. Sampling rate counts amplitude measurements per second. A 1 kHz component has 44.1 measurements per cycle at Fs=44.1 kHz.

2. 11,025 Hz. Suppress frequencies that the lower rate cannot represent with anti-aliasing filtering before discarding samples. A practical transition region is needed.

3. At Fs=22,050 Hz, the retained samples cannot uniquely distinguish those components. For the illustrated cosines, 22,050-14,000=8,050 Hz gives the same values at the retained sample times.

4. Positions 1,3,5,... . For five input samples, three remain: ceil(5/2). MATLAB indexing is one-based.

5. Odd input lengths round up when halved. Dividing the retained count by the halved rate therefore differs slightly from the original N/Fs. The difference is less than one output sample period.

6. resample includes low-pass anti-aliasing processing and compensates its designed filter delay. Direct indexing simply discards samples. Both outputs must be associated with Fs/2.

7. [0,1000,2000,3000,4000] Hz. Bin spacing is Fs/N=1,000 Hz. There are five retained entries because the zero-frequency/DC entry is included.

8. For even N, double interior positive-frequency magnitudes, leaving DC and Nyquist unchanged. For odd N, there is no exact Nyquist bin, so double everything after DC through the last retained entry.

9. -60 dB has ten times the amplitude of -80 dB. The values are levels below a positive reference amplitude of 1; negative dB does not mean a negative amplitude.

10. Horizontal bands suggest sustained components; vertical streaks suggest brief broadband events. Resolving nearby components also depends on observation/window duration and window shape; padding adds frequency-grid points rather than new observed signal information.

> **Ready to progress?**
> You should be able to trace a sample index into a time and an FFT-bin number into a frequency. If those mappings are unclear, revisit pages 3, 8 and 12 before reviewing the model.

Answer key B

Use the quantities and assumptions from the current 22.05 kHz case.

11. Start with 882,005 samples. Retain ceil(N/2)=441,003. Reconstruct at twice that rate to get 882,006. Keep the first 882,005 positions because those are the original time-grid positions.

12. Interpolation adds estimates between available sample values. Frequencies removed by filtering are absent, and aliased components are already indistinguishable from genuine lower frequencies.

13. 1,024 samples/frame, one channel, 431 frames. Total values: 1,024*431=441,344. (:)-indexing joins the frames in order for this logged layout.

14. 1,024 samples arrive in every frame interval. Thus Fs_audio=1,024/framePeriod=22,050 samples/s. The reciprocal frame period alone counts frames per second.

15. Raw delay=(41-1)/2=20 input samples, equivalent to 10 output samples. MATLAB resample has compensated it; the raw Simulink FIR output has not. Starting at 11 skips those 10 delayed samples.

16. It establishes close implementation agreement for matching settings/times. Both outputs can agree while both differ substantially from the original. Original-versus-processed NMSE/SNR is a separate calculation.

17. The MAT file retains the double arrays. The WAV uses a finite 16-bit PCM grid, so writing and reading can change the filtered values slightly. The export error is recorded separately.

18. Sampling_Abdulrahman uses the input recording and its own calculations. Compare_Sampling additionally reads out.simNoFilter and out.simFiltered, which exist only after the model runs.

19. All original sample times on the 44.1 kHz grid, starting at zero; only unmatched endpoint/frame coverage and known causal delay are removed where applicable. Extra error-based trimming or optimized time shifts are not part of the agreed policy.

20. Examples to ignore: slprj/, *.slxc, *.asv, *.slx.original. Examples to track: .m source, .slx source models, the selected original WAV, intentional MAT/WAV results and task documents.

> **Three facts worth saying clearly**
> A lower sampling rate has a lower Nyquist boundary. An implementation can match another implementation while degrading the original. Correct time alignment changes the fairness of the measurement, not the information lost during processing.

Source map and quick glossary

The guide follows the repository, with official documentation for API behavior.

Term	Short meaning
Aliasing	Different physical frequencies become indistinguishable after sampling.
Decimation	Reduce rate by an integer factor; anti-alias filtering is needed for a properly band-limited result.
Frame	A block of consecutive audio samples delivered together.
Group delay	Time displacement caused by filter phase; constant for this symmetric linear-phase FIR.
Reconstruction	Interpolation to a chosen time grid; not recovery of lost information.
PCM	A digital audio sample representation with a fixed set of numeric levels.

Repository evidence

[R1] Matlab/FirstPart_AudioFreqAnalysis.m and Simulink/FirstPart_TimeFreqSimulink.slx.
[R2] Matlab/Sampling_Abdulrahman.m; Matlab/Compare_Sampling.m; Simulink/SamplingModel_Abdulrahman.slx.
[R3] Results/Abdulrahman/AA02/summary.csv and sampling_22050.mat; AA03/known_tone_validation.csv; team/Abdulrahman.md.
[R4] CCEN 302 Project-3.pdf, especially requirements/workflow and deliverables.

Open the verified repository checkpoint: be17af9

Official MathWorks references

[S1] resample: rate factors, FIR design and delay compensation
[S2] kaiser: window shape parameter
[S3] fft: transform and one-sided spectrum examples
[S4] spectrogram: short-time Fourier analysis
[S5] Downsample block
[S6] FIR Decimation block
[S7] To Workspace: formats and simulation output
[S8] audiowrite: WAV precision and valid ranges

Documentation consulted 25 October 2026; installed code/model verification used MATLAB R2026a. Some live documentation displays R2026b. The source checkpoint defines the exact implementation discussed. No future-rate or full-quality results are invented in this guide.